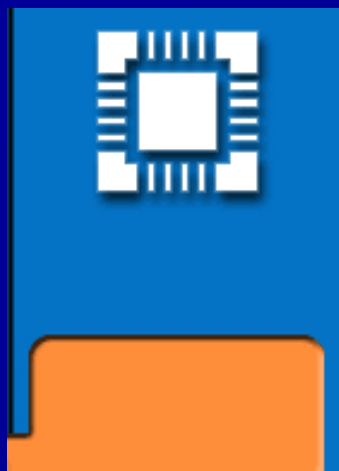
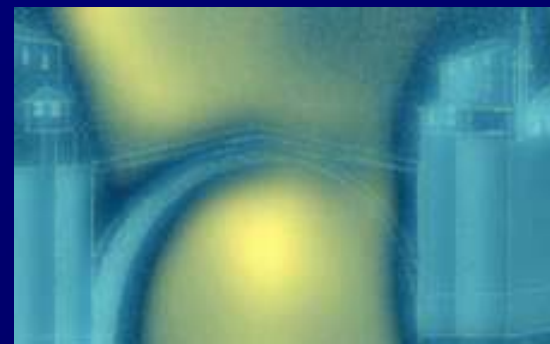




Studij informatike

Algoritmi i strukture podataka



Općenito o predmetu

- U školskoj godini 1999/2000 odlučili smo modificirati izvođenje nastave iz ovog predmeta. Umjesto klasičnih folija gradivo će se prezentirati računarm i to koristeći alat PowerPoint, rad programa će se izravno demonstrirati izvođenjem u Visual C++ okruženju. I tekst i programski kod mogu se stalno poboljšavati. Dobar dio do sada obavljenih ispravki i poboljšanja dugujemo upozorenjima studenata prethodnih generacija te im ovom prilikom zahvaljujemo. Sigurno još uvijek ima i pogrešaka i nedostataka, a zbog uvedenih promjena, sigurno su nastale i nove te molimo studente da nam na njih ukažu.
- Laboratorijske vježbe održavat će se po 2 sata tjedno, ali počevši od 08. studenog 1999. Isključeni su tjedni od 20.12.1999 - 09.01.2000. To znači da ima 7 tjedana + 1 tjedan za nadoknade (**PROVJERITI!**).

Općenito o predmetu

- Pohađanje nastave **neće se evidentirati i nije uslov** za dobivanje potpisa. Međutim:
- Student koji nema vlastite bilješke s nastave **nema pravo dolaska na konzultacije**. Vlastita teka s bilješkama, odnosno dopunama i komentarima na nastavnim materijalima je nužan uslov za bilo kakvo osobno obraćanje nastavnicima ovog predmeta izvan predavanja ili vježbi. Dio studenata uobičajeno smatra da ne mora pohađati nastavu. To smatramo njihovim pravom (koje je na njihovu štetu!), ali pri tome moraju ostati konzekventni!
- Pretpostavka ovog predmeta je operativno znanje programiranja u jeziku C. Studenti, koji su eventualno uspjeli položiti Programiranje ne naučivši C, imat će ozbiljnih problema s polaganjem ovog ispita. Stoga ih upozoravamo da manjkavosti u svom znanju pravovremeno nadoknade. Zahtjevi na znanje iz ovog predmeta povećat će se proporcionalno našoj percepciji poboljšanja izvedbe ovog predmeta.

Općenito o predmetu

- Predmet je uveden u nastavni plan kao zajednički odlukom oba studija i svih smjerova, a ne na osnovu nastojanja predmetnih nastavnika (koji su predmet predložili samo za Računarstvo). Naša je obveza poštivati ovu odluku Fakulteta te studentima ne možemo davati nikakve olakšice temeljem pozivanja na odabir smjera koji tobože ne treba ovo gradivo. Argumenti da je predmet nekome uslov za upis na treću ili čak četvrtu godinu, prijetnja gubitka prava studija zbog 8. pada na ispitu i slično neće utjecati na kriterije ispitivača. Naprotiv, takve situacije ukazuju da dotični student najvjerojatnije studira na pogrešnom fakultetu! Ako netko usprkos tome dođe zamoliti za "razumijevanje", znači da čak nije upoznat niti s ovim uvodnim poglavljem. Kriterije ćemo dakle nastojati zadržavati jednakima bez obzira na karakter ispitnog roka.

Općenito o predmetu

- uslov za potpis: Kolokvirane laboratorijske vježbe
- Ispit: Riješeni zadatak za domaći rad, pismeni i usmeni ispit
- Konzultacije (**donijeti vlastite zabilješke s nastave!**):

Prof.dr.sc. Damir Kalpić (D-368)

Prof.dr.sc. Vedran Mornar (D-366)

Svaki dan od 12-13h

ZPM, Zgrada D/III kat, sjeverozapadno krilo

- Sve primjedbe na predmet i oko njega šaljite na:

`damir.kalpic@fer.hr` ili `vedran.mornar@fer.hr`
SUBJECT:ASP

Općenito o predmetu

■ Literatura

- Kalpić & Mornar: Materijali s predavanja
- <http://www.zpm.fer.hr/courses/algor>
- Vlastita teka
- Budin: Informatika za 1. razred gimnazije, Element, Zagreb, 1996
- Horowitz & Sahni: Fundamentals of Computer Algorithms, Pitman, London, 1978 (ima i novo izdanje!)
- Knuth: Fundamental Algorithms, Addison-Wessley, 1973
- Wirth: Algorithms + Data Structures = Programs, Prentice-Hall, 1976
- Weiss: Data Structures and Algorithm Analysis in C, Addison Wesley, 1997

Uvod

Algoritam

■ Što je algoritam?

- Abu Ja'far Mohammed ibn Musa al Khowarizmi (ili al Horezmi, u latinskim tekstovima citiran kao "Dixit Algorizmi" -> Algorithmus) (oko 825. god.)
- Precizno opisan način rješenja nekog problema
- Jednoznačno određeno šta treba napraviti
- Moraju biti definirani početni objekti koji pripadaju nekoj klasi objekata na kojima se obavljaju operacije
- Kao ishod algoritma pojavi se završni objekt(i) ili rezultat(i).
- Konačni broj koraka; svaki korak opisan instrukcijom
- Obavljanje je algoritamski proces
- upotrebljiv, ako se dobije rezultat u konačnom vremenu

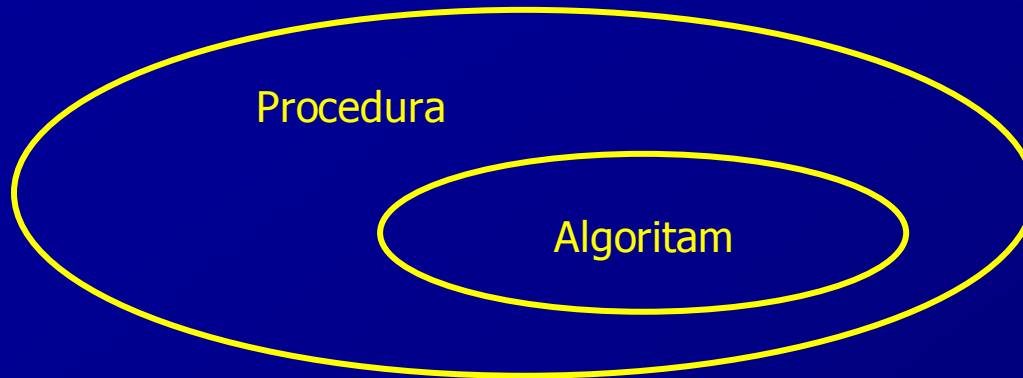
■ Primjeri za nedopuštene instrukcije:

- izračunaj $5/0$
- uvećaj x za 6 ili 7

Algoritam

- efikasnost:
 - U konačnom vremenu može se dobiti rezultat koristeći olovku i papir.
- Primjeri:
 - Zbrajanje cijelih brojeva je efikasno
 - Zbrajanje realnih brojeva nije jer se može pojaviti broj s beskonačno mnogo znamenki

Procedura



- Postupak koji ima sva svojstva kao i algoritam, ali ne mora završiti u konačnom broju koraka jest računarska procedura. U jeziku C to može biti `void` funkcija
- Primjeri za proceduru:
 - Operacijski sistem računala
 - Uređivač teksta
- Vrijeme izvođenja mora biti "razumno"
- Primjer:
 - Algoritam koji bi izabirao potez igrača šaha tako da ispita sve moguće posljedice poteza, zahtijevao bi milijarde godina na najbržem zamislivom računalu.

Algoritmi i programi

- *Program* - Opis algoritma koji u nekom programskom jeziku jednoznačno određuje šta računar treba napraviti.
- *Programiranje* - naučiti sintaksu nekog proceduralnog jezika i steći osnovna intuitivna znanja u pogledu algoritmizacije problema opisanog riječima.
- Algoritmi + strukture podataka = PROGRAMI (*Wirth*)
 - kako osmisлити algoritme
 - kako strukturirati podatke
 - kako formulirati algoritme
 - kako verificirati korektnost algoritama
 - kako analizirati algoritme
 - kako provjeriti (testirati) program
- Postupci izrade algoritama nisu jednoznačni te zahtijevaju i kreativnost. Inače bi već postojali generatori algoritama. Znači da se (za sada?) gradivo ovog predmeta ne može u potpunosti algoritmizirati . Koristit će se programski jezik C. Za sažeti opis složenijih algoritama može se koristiti pseudokod.

Statické struktury podataka

Osnovni tipovi

- `char` - znakovni tip (1 By)
- `int` - cjelobrojni tip (2-4 By)
- `float` - realni tip (4 By)
- `double` - realni tip u dvostrukoj preciznosti (8 By)
 - Razlika je između preciznosti (*precision*) i tačnosti (*accuracy*). Preciznost se iskazuje brojem prvih važećih tačnih znamenki, a tačnost je bliskost stvarnoj (nepoznatoj) vrijednosti. Za dovoljnu tačnost potrebna je adekvatna preciznost, ali preciznost ne implicira automatski tačnost jer su iskazane znamenke mogle nastati na osnovu npr. pogrešnog mjerenja.

Prefiksi ili kvalifikatori

- Odnose se na cijele brojeva. Duljina ovisi o procesoru.
- `short` - smanjuje raspon vrijednosti koje varijabla može sadržavati (2 By)
- `long` - povećava raspon vrijednosti koje varijabla može sadržavati (4 By)
- `signed` - dozvoljava pridruživanje pozitivnih i negativnih vrijednosti
- `unsigned` - dozvoljava pridruživanje samo pozitivnih vrijednosti

Memorijske klase

- `memorijska_klasa` utvrđuje postojanost (trajnost) i područje važenja varijable ili polja u memoriji ovisno o mjestu deklaracije u programu.
- Postoje 4 memorijske klase:
 - `auto` automatska (vrijedi lokalno unutar funkcije)
 - `extern` vanjska (vrijedi globalno unutar programa)
 - `static` statička (vrijedi lokalno unutar funkcije ili modula)
 - `register` registarska (vrijedi lokalno unutar funkcije, ali koristi CPU registre)
- Obično se ključna riječ `auto` ne navodi, te su sve lokalne varijable i polja definirani unutar neke funkcije automatske klase. Vanjska klasa ukazuje na varijable i polja koji su globalni (zajednički) za sve funkcije unutar programa i obično se `extern` ne navodi jer položaj izvan funkcije ukazuje na to.
- Statička klasa se koristi onda kada se vrijednost varijable ili članova polja treba zadržati nakon izlaska i ponovnog povratka u neku funkciju.
- U opisu algoritama izbjegavat će se globalne varijable da bi se eksplicitno ukazalo na razmjenu informacija među funkcijama.

Niz znakova; Logička vrijednost

■ Niz znakova

Z	a	g	r	e	b	\0
---	---	---	---	---	---	----

- Numerička vrijednost 0 oznaka je kraja znakovnog niza.

```
char ime_niza[duljina_niza+1];
```

■ Logička vrijednost

- U nekim jezicima postoji poseban tip podataka `LOGICAL`.
- U C-u se svaki tip podatka može koristiti kao logički.

```
#define TRUE 1
```

```
#define FALSE 0
```


Polje

- Polje je *struktura podataka* gdje isto ime dijeli više podataka
- Svi podaci u nekom polju moraju biti istog tipa i iste *memorijske klase*
- Elementi (članovi) polja se identificiraju po *indeksom*
- Indeks može biti nenegativni cijeli broj (konstanta, varijabla, cjelobrojni izraz)

`x[0]` `x[9]` `x[n]` `x[MAX]` `x[n+1]` `x[k/m+5]`

- Polje može biti
 - jednodimenzionalno (vektor)

```
#define N 100  
float x[N];
```



`x[0]` `x[1]` `x[2]` `...` `x[n-2]` `x[n-1]`

Polje

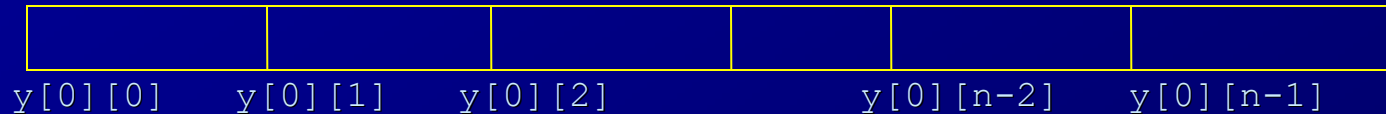
- dvodimenzionalno (matrica, tablica)

```
#define N 100
```

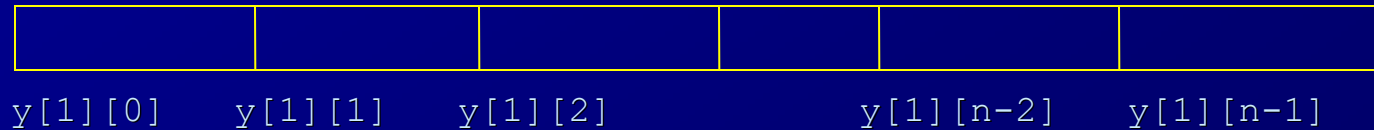
```
#define M 50
```

```
float y[M][N];
```

red 1

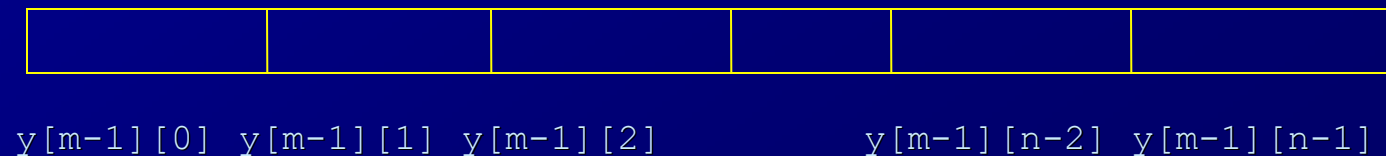


red 2



• • •

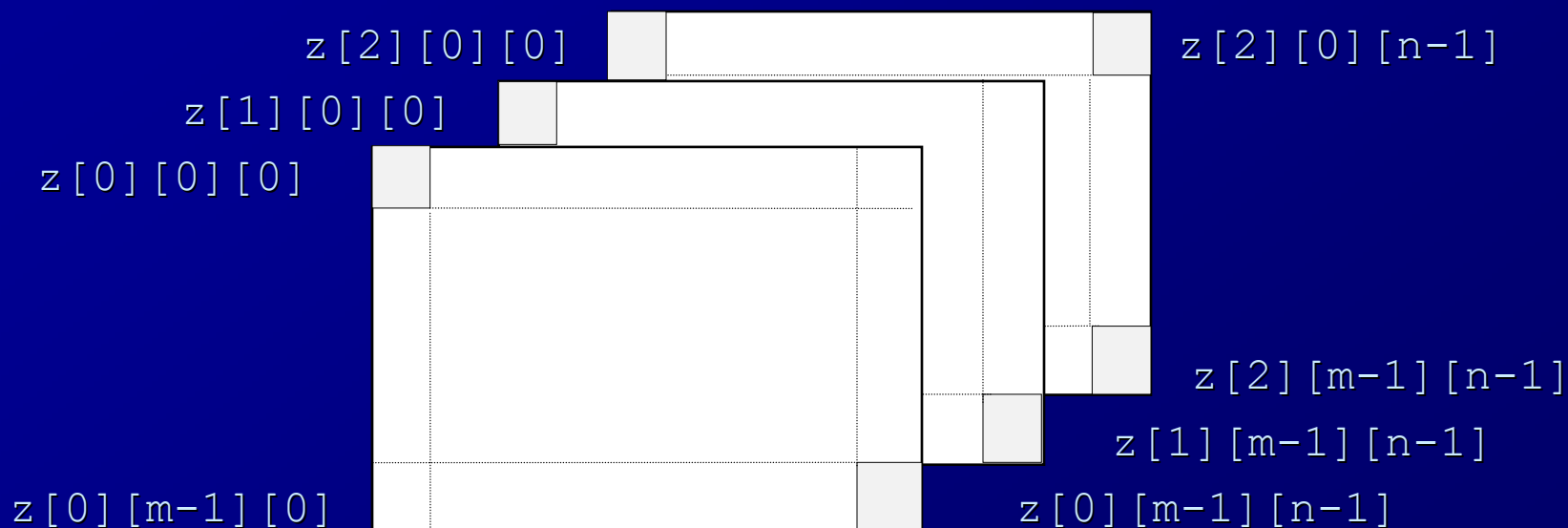
red m



Polje

- trodimenzionalno i višedimenzionalno

```
# define N 100  
# define M 50  
float z[3][M][N];
```

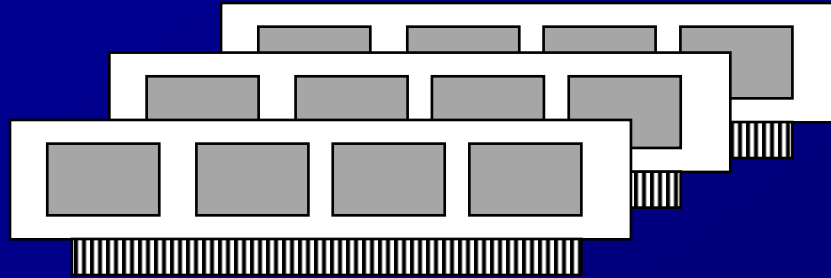


- Opći oblik naredbe za deklaraciju polja:

```
memorijska_klasa tip_podatka polje[izraz1][izraz2]...
```

Pokazivač (Pointer)

Memorija računara:



zapravo je kontinuirani niz bajtova:



0 1 2 3 4...

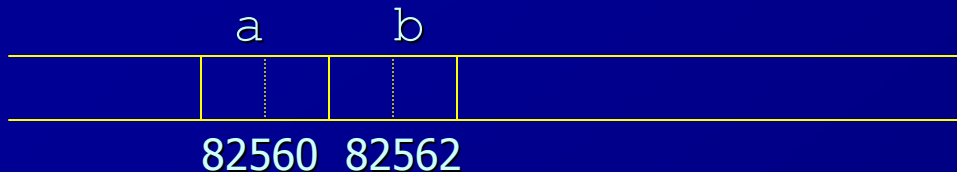
64-128MB

Svaki bajt ima svoj redni broj: *adresa*

Deklaracija "običnih" varijabli i pokazivača

- Deklaracijom varijabli rezervira se prostor u memoriji na slobodnim adresama, npr.:

```
short a, b;
```



- Deklaracijom pokazivača rezervira se prostor u memoriji u duljini 4 bytea kako bi se pohranila bilo koja adresa u adresnom prostoru od 4GB, npr.:

```
short *p;
```

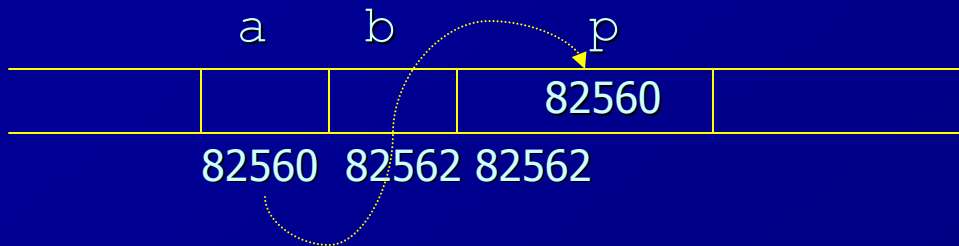


- Važno je primijetiti da deklaracijom ni jednoj od varijabli nije inicijalizirana vrijednost

Varijable i pokazivači

- Vrijednost pokazivača svakako treba postaviti prije upotrebe

`p = &a;`



- Ovime je ostvareno pokazivanje pokazivača `p` na varijablu `a`



- Sada je moguće indirektno postaviti vrijednost varijable `a`

`*p = 7;`



Varijable i pokazivači

- Indirektno se može i koristiti vrijednost varijable a

b = *p;



IndirektnaZamjenaVrijednosti

- Vrijednost pokazivača može se postaviti i rezervacijom slobodne memorije

PrimjerZaMalloc

```
main () {  
    short int *p;  
    p = (short int *) malloc (sizeof (short int));  
    *p = 7;  
}
```

- Valja razlikovati:

- p pokazivač veličine 4 bajta na broj tipa short int
- *p broj tipa short int veličine 2 bajta. Ne mora postojati!

Aritmetika s pokazivačima

- Aritmetika s pokazivačima podrazumijeva korištenje jedinica koje odgovaraju duljini (By) strukture podataka na koju pokazivač pokazuje.
- Uvećanje za 1 pokazivača na strukturu dugačku 4 By znači da se njegova vrijednost uvećava za 4. Ako je struktura dugačka 8 By, uvećanje pokazivača za 1 povećava vrijednost pokazivača za 8 itd.

● Primjer:  AritmetikaPokazivaca

(long = 4 By, double = 8 By)

```
long dugi; double dupli;  
long *pdugi; double *pdupli;  
pdugi = &dugi;  
pdupli = &dupli;  
++pdugi;  
pdupli = pdupli + 2;
```

Vrijednosti

<u>pdugi</u>	<u>pdupli</u>
?	?
128560	?
128560	128564
128564	128564
128564	128580

Polja i pokazivači

📁 PoljeJePokazivac

```
#include <stdio.h>
```

```
main () {
```

```
    int x[4] = {1,2,3,4};
```

```
    printf ("%d %d\n", *x, *(x+1));
```

```
    f (x);
```

```
}
```

```
void f (int *x) {           ili          void f (int x[]) {
```

```
    printf ("%d %d\n", *x, x[0]);
```

```
    ++x;
```

```
    printf ("%d %d %d\n", *x, x[0], *(x-1));
```

```
}
```

Ispis na ekranu:

1 2

1 1

2 2 1

x[0] x[1] x[2] x[3]

	1	2	3	4	
--	---	---	---	---	--

Zapisi (strukture)

■ Typedef deklaracija

```
typedef stari_tip novi_tip;
```

npr.

```
typedef unsigned size_t;
```

```
typedef int redni_broj;
```

```
typedef short logical;
```

```
redni_broj i, j;
```

```
size_t velicina;
```

```
logical da_ne;
```

■ Definiranje strukture

- Strukture podataka čiji se elementi razlikuju po tipu:

```
struct ime_strukture {
```

```
    tip_elementa_1 ime_elementa_1;
```

```
    tip_elementa_2 ime_elementa_2;
```

```
    ...
```

```
    tip_elementa_n ime_elementa_n;
```

```
};
```

Zapisi (struktura)

- Primjer

```
struct osoba {  
    char jmbg[13+1];  
    char prezime[40+1];  
    char ime[40+1];  
    int visina;  
    float tezina;  
};
```

Ovime nije deklariran konkretan zapis, već je samo definirana struktura zapisa.

Deklaracija konkretnih zapisa:

```
struct ime_strukture zapis1, zapis2, ... , zapisN;
```

npr.

```
struct osoba o1, o2, zaposleni[500];
```

Zapisi (strukture)

- Moguće je definiranje statičke strukture podataka proizvoljne složenosti jer pojedini element može također biti `struct`:

```
struct student {  
    int maticni_broj;  
    struct osoba osobni_podaci;  
    struct adresa adresa_roditelja;  
    struct adresa adresa_u_Zagrebu;  
    struct osoba otac;  
    struct osoba majka;  
};
```

Alternativno, korištenjem naredbe `typedef`:

```
typedef struct {  
    char jmbg[13+1];  
    char prezime[40+1];  
    char ime[40+1];  
    int visina;  
    float tezina;  
} osoba;  
osoba o1, o2, zaposleni[500];
```

Zapisi (strukture)

```
typedef struct {  
    int maticni_broj;  
    osoba osobni_podaci;  
    adresa adresa_roditelja;  
    adresa adresa_u_Zagrebu;  
    osoba otac;  
    osoba majka;  
} student;
```

■ Referenciranje elemenata zapisa

```
zapis.element = vrijednost;  
vrijednost = zapis.element;  
npr:  
student pero;  
pero.majka.tezina = 92.5;
```

Zapisi (struktura)

■ Pokazivač na strukturu

```
struct osoba *p;
```

*p - zapis o osobi

p - adresa zapisa o osobi

- Referenciranje na element strukture preko pokazivača

p->prezime ili (*p).prezime

 PrimjerZaStruct

Elementi izrade programa

Normalan slijed; Bezuslovni skok

- Normalan programski slijed

naredba_1

naredba_2

naredba_3

...

- Bezuslovni skok

Pseudokod:

idi na *oznaka_naredbe*

C:

`goto` *oznaka_naredbe;*

Grananje

- S: Oznaka za jednu ili više naredbi, odnosno programski odsječak
- uslovno obavljanje naredbi (jednostrana selekcija)

Pseudokod: C:

ako je (*logički_izraz*) onda
| *S*

```
if (logički_izraz) {  
    S;  
}
```

- Grananje (dvostrana selekcija)

Pseudokod: C:

ako je (*logički_izraz*) onda
| *S_1*
inače
| *S_2*

```
if (logički_izraz) {  
    S_1;  
} else {  
    S_2;  
}
```

Grananje

■ Višestruko grananje (višestрана selekcija)

Pseudokod:

```
ako je (logički_izraz_1) onda  
  | S_1  
inače ako je (logički_izraz_2) onda  
  | S_2  
inače ako je (logički_izraz_3) onda  
  | S_3  
  
  ...  
  
inače  
  | S_0
```

C:

```
if (logički_izraz_1) {  
  S_1;  
} else if (logički_izraz_2) {  
  S_2;  
} else if (logički_izraz_3) {  
  S_3;  
  
  ...  
  
} else {  
  S_0;  
}
```

Grananje

■ Skretnica

Pseudokod:

skretnica (*vrijednost*)

slučaj *C1*

| *S_1*

slučaj *C2*

| *S_2*

...

slučaj *Cn*

| *S_n*

inače

| *S_{n+1}*

C:

switch (*cjelobrojna vrijednost*) {

case *C1*:

S_1;

break;

case *C2*:

S_2;

break;

...

case *Cn*:

S_n;

break;

default:

S_{nplus1};

}

Grananje

- Primjer za vježbu: Ostvariti skretnicu naredbom `if - else`.

```
if (vr == C1) {  
    S1;  
} else if (vr == C2) {  
    S2;  
}  
  
...  
} else if (vr == Cn) {  
    Sn;  
} else {  
    S_nplus1;  
}
```

- Ako se iz skretnice izbace naredbe `break`, obavljaju se slijedno sve naredbe iza one gdje je prvi put zadovoljeno: `vr == Ci`

Grananje

- Skretnica bez `break` korištenjem naredbe `if`.

```
nadjen = 0;
if (vr == C1) {
    S1;
    nadjen = 1;
}
if (vr == C2 || nadjen) {
    S2;
    nadjen = 1;
}
...
if (vr == Cn || nadjen) {
    Sn;
}
S_nplus1;
```

Programska petlja

- Petlja s ispitivanjem uslova ponavljanja na početku

<u>Pseudokod:</u>	<u>C:</u>
<u>dok je</u> (<i>logički_izraz</i>) <u>činiti</u> <i>S</i>	<code>while (<i>logički_izraz</i>) {</code> <code> <i>S</i>;</code> <code>}</code>

- Petlja s ispitivanjem uslova ponavljanja na kraju

- U nekim programskim jezicima postoji oblik: REPEAT...UNTIL

<u>Pseudokod:</u>	<u>C:</u>
<u>ponavljati</u> <i>S</i> <u>do</u> (<i>logički_izraz</i>)	<code>do {</code> <code> <i>S</i>;</code> <code>} while (!<i>logički_izraz</i>)</code>

- Standardna petlja u C-u:

<u>Pseudokod:</u>	<u>C:</u>
<u>ponavljati</u> <i>S</i> <u>dok je</u> (<i>logički_izraz</i>)	<code>do {</code> <code> <i>S</i>;</code> <code>} while (<i>logički_izraz</i>);</code>

Programska petlja

■ Petlja s poznatim brojem ponavljanja

<u>Pseudokod:</u>	<u>C:</u>
<u>za</u> $i := poc$ <u>do</u> $kraj$ (<u>korak</u> k) <u>činiti</u> S	<code>for (i=poc; i<=kraj; i=i+k) {</code> $S;$ <code>}</code>

● Primjer: Realizacija istog odsječka petljom `while`:

```
i = poc;
while (i <= kraj) {
     $S;$           // niz naredbi koje ne mijenjaju vrijednost za i
    i += k;
}
```

ili općenitije:

```
i = poc;
while ((i - kraj) * k <= 0) {
     $S;$           // niz naredbi koje ne mijenjaju vrijednost za i
    i += k;
} // U čemu je razlika? Pokus: poc = 5; kraj = -20; k = -7;
```

Programska petlja

■ Skok iz petlje

Pseudokod:

izađi iz petlje

skoči na kraj petlje

C:

`break;`

`continue;`

■ Beskonačna petlja

Pseudokod:

ponavljaj

| *S*

zauvijek

C:

`while(1) {`

S;

`}`

- U algoritmima ovakva petlja nije dopuštena jer je u suprotnosti sa zahtjevom da postupak bude konačan.

Programska petlja

- U tijelu petlje mora postojati barem jednom ispitivanje uslova za izlazak iz petlje:

```
while(1) {
```

```
    S1;
```

```
    if (logički_izraz) break;
```

```
    S2;
```

```
    ...
```

```
}
```

ili

```
while(1) {
```

```
    S1;
```

```
    if (logički_izraz) goto oznaka_naredbe;
```

```
    S2;
```

```
    ...
```

```
}
```

- Naredbu `goto` treba izbjegavati ako je ikako moguće. Smanjuje se mogućnost pogreške i olakšava testiranje programa ako svaka programska cjelina (npr. petlja) ima samo jedan ulaz i jedan mogući izlaz.

Procedure

- Programi se sastoje od procedura. Prva pozvana procedura je glavni program. Kad glavni program završi, slijedi povratak u operativni sistem. Inače, povratak iz procedure je uvijek na onu proceduru koja ju je pozvala. Uobičajena je podjela na funkcije (*function*) koje imaju od nula do više ulaznih argumenata i vraćaju jedan rezultat, te na općenite potprograme (*subroutine*) koje rezultat predaju argumentima.
- U jeziku C sve procedure su funkcije koje daju rezultat nekog od tipova podataka, ali mogu mijenjati i vrijednost argumenata.
- Posebni slučajevi:
 - Glavni program: `main`
 - Potprogram koji u imenu ne vraća vrijednost: `void`

Razmjena podataka između funkcija

- globalne varijable
- argumenti navedeni u zagradi, prenose se vrijednosti (*call by value*)
- za prijenos vrijednosti u pozivajuću funkciju koriste se kod poziva funkcije kao argumenti adrese, a u definiciji funkcije argumenti su pokazivači (*call by reference*).
- Ako funkcija mora predati rezultat preko argumenata, obavezno se koristi *call by reference*.
- Primjer: zamjena vrijednosti dvije cjelobrojne varijable
 📁 KomunikacijaSFunkcijama

Ulazno-izlazne operacije

- Za slijedno čitanje/pisanje preko standardnih ulazno-izlaznih jedinica koristit će se odgovarajuće C funkcije ili naredbe pseudokoda:
 - ulaz (*lista adresa argumenata*)
 - izlaz (*lista argumenata*)
- Kod čitanja je dakle nužan *call by reference*, dok kod ispisa može poslužiti i *call by value*.

Dvodimenzionalna i višedimenzionalna polja kao argumenti funkcije

- Polje u funkciji treba dočekati kao jednodimenzionalno polje ili pokazivač.

- Primjer:

```
int polje[3][4] = { { 1, 2, 3, 4},  
                   { 5, 6, 7, 8},  
                   { 9, 10, 11, 12} };
```

u memoriji računara spremljeno je kao

	1	2	3	4	5	6	7	8	9	10	11	12	
--	---	---	---	---	---	---	---	---	---	----	----	----	--

tj. jednako kao jednodimenzionalno polje od 12 elemenata.

- Razlikuju se fizičke dimenzije polja od logičkih, koje mogu biti i manje. Za pozicioniranje je potrebno znati fizički broj stupaca (`MAX_broj_stupaca`).
- Za dohvat elementa iz i -tog reda treba preskočiti $i-1$ punih redova.
- Kako prvi red ima indeks 0, drugi 1, treći 2 itd., za dohvat elemenata retka s indeksom i treba preskočiti $i * \text{MAX_broj_stupaca}$ članova polja.
- Općenito:

`dvodim_polje[i][j] ≡ jednodim_polje[i * MAX_broj_stupaca + j]`

- Primjer korištenja dvodimenzionalnog polja u potprogramu:

 `DvodimenzionalnoPolje`

Pisanje strukturiranih programa

■ Osnovni naputci

- specifikacija ulaznih i izlaznih varijabli (u C obvezatno!)
- definicija lokalnih varijabli
- tok programa prema dolje, osim petlji i neizbježne goto naredbe
- poravnati naredbe istog nivoa za povećanje čitkosti (*indentation*)
- dokumentacija kratka ali smisljena
- koristiti potprograme gdje je to primjereno

■ Izbor vrste petlje

- Primjer: Čitati članove nekog skupa nenegativnih brojeva sve dok njihova suma ne premaši neki predviđeni iznos n .

```
y = 0;
```

```
do {
```

```
    ulaz(&x);
```

```
    y += x;
```

```
} while (y <= n);
```

```
y = 0;
```

```
while (y <= n) {
```

```
    ulaz(&x);
```

```
    y += x;
```

Programi nisu funkcionalno identični jer u prvom slučaju se uvijek obavlja barem jedno čitanje, pa program ispravno radi samo za $n \geq 0$. U drugom slučaju, ako je $n < 0$, smatra se da je bez čitanja ijednog podatka postupak završen.

Pisanje strukturiranih programa

- Primjer: Pročitati n vrijednosti i obraditi ih procedurom PROCES. Korištenjem petlje while:

```
i = 0;
while (i < n) {
    ulaz(&x);
    PROCES(x);
    i++;
}
```

S obzirom da je unaprijed poznat broj ponavljanja, primjerenije je koristiti petlju s poznatim brojem ponavljanja:

```
for (i = 0; i < n; i++) {
    ulaz(&x);
    PROCES(x);
}
```

Ne radi se samo o manjem broju naredbi, nego se smanjuje i mogućnost greške u pisanju programa.

Pisanje strukturiranih programa

- Primjer: Treba pročitati i obraditi skup podataka sve dok ne naiđe oznaka kraja podataka (EOF). Realizacija petljom `while`:

```
ulaz (&x) ;  
while (x != EOF) {  
    PROCES (x) ;  
    ulaz (&x) ;  
}
```

Treba dakle pročitati prvi podatak, obrađuje ga se samo ako postoji, a nakon obrade čita se sljedeći podatak koji se obradi u idućem prolasku kroz petlju.

Bolje je rješenje skokom iz petlje:

```
while (1) {  
    ulaz (&x) ;  
    if (x == EOF) break ;  
    PROCES (x) ;  
}
```

Ne narušava se željeni princip da iz jednog programskog bloka postoji samo jedan izlaz.

Pisanje strukturiranih programa

- Ako ima dva kriterija za završetak petlje: kraj podataka ili da rezultat procedure PROCES, pohranjen u varijabli `y` bude nula:

```
do {  
    ulaz(&x);  
    if (x == EOF) break;  
    PROCES(x, &y);  
} while (y != 0);  
if (x != EOF) {  
    ...  
} else {  
    ...  
}
```

Na izlasku iz programskog segmenta ne zna se zbog čega je petlja završila pa se to mora ispitivati.

■ Korišćenje naredbe `case`

- Uvijek se može upotrijebiti elementarnija naredba `if-else`. Naredba `case` je u prednosti kad ima mnogo ravnopravnih grananja.

Višeobličje; Logičke operacije

■ Razlika između opisa algoritma i realizacije - višeobličje

- Deklarirat će se tip varijable tamo gdje je to jednoznačno za algoritam. C ne podržava višeobličje (polimorfizam). Umjesto nekog od elementarnih tipova može se (zahvaljujući naredbi `typedef`), koristiti općeniti *tip*.
- Primjer: Traženje najvećeg člana u jednodimenzionalnom polju

 `MaxClanStd`

■ Obavljanje logičkih operacija

- U analizi algoritama pretpostavljat će se obavljanje logičkih operacija na najkraći način (reducirano ispitivanje, *short-circuit evalation*). Npr. logički izraz $(\text{logički_izraz1} \vee \text{logički_izraz2})$ daje odmah rezultat *true* ako je *logički_izraz1* = *true*, bez vrednovanja za *logički_izraz2*. Analogno $(\text{logički_izraz1} \wedge \text{logički_izraz2})$ daje odmah rezultat *false* ako je *logički_izraz1* = *false*.